WPT Day 6 Evening

Mon 26 Oct 26 · ECMAScript as the JS blueprint, Node.js as a runtime, built-in and user-defined modules, `fs` file I/O, a local HTTP server, and sockets (IP + port)

Table of Contents

1. ECMAScript: What It Is and Why It Exists
2. Node.js: Runtime, Installation and First Run
3. Modules: Concept, Types and require
4. fs Module: Reading and Writing Files
5. HTTP Module: Building a Local Server
6. Custom Modules: Creating and Exporting
7. Combined Worked Example: Math Module + File Log + HTTP Server
8. Quick Revision Checklist
9. Practice Exercises
10. Practice Answers

ECMAScript: What It Is and Why It Exists

The problem ECMAScript solves

JavaScript looks like "just a language", but historically it was not one agreed-upon language. Without a shared rulebook, two browsers could implement "JavaScript" differently, and the same source file would behave differently — or fail — depending on which browser opened it. That is the problem ECMAScript exists to solve.

Mechanism: specification vs implementation

ECMAScript is a specification (a blueprint), not a language itself. It defines the rules, regulations, syntax, and features that a conforming JavaScript engine must follow. JavaScript is the *implementation* of that blueprint. ECMA (the organisation) publishes the specification; engines and runtimes implement it.

The lecture's one-line framing is worth memorising:

> "ECMAScript is just a blueprint for JavaScript."

Consequences that follow directly from that definition:

- Every new JS feature **originates in ECMAScript**, not in "JavaScript inventing something on its own".

- Arrow functions, classes, inheritance syntax, promises, template literals — all of these entered the language because ECMAScript specified them, and JavaScript engines then implemented those rules.
- New ECMAScript versions ship roughly every year. The lecture notes the latest as **ECMAScript 2026**, with **ES 2027** upcoming.

Background: "ES6" is the historic nickname for the major 2015 release (ECMAScript 2015). After that, yearly releases are often called ES2016 / ES2017 / … or ES7 / ES8 / … informally. When someone says "ES6+ features", they mean the modern language surface that began with that big release.

Why ECMAScript was needed: the browser wars

In the early 1990s, **Netscape** developed **LiveScript** so web pages could be interactive. **Microsoft** saw the success and built its own dialect: **JScript**.

Piece	Netscape world	Microsoft world
Dialect	LiveScript	JScript
Who could run it	Netscape's browser	Microsoft's browser
Developer cost	Write once for Netscape	Write again for IE

Result: developers had to maintain **separate code for each browser**. The lecture's phrasing: "Developers' lives became a nightmare." Users rarely saw the pain directly; bugs were raised against developers who had to paper over incompatibilities.

Around **1996–97**, **ECMA** stepped in to unify the ecosystem:

- Declared that browsers must follow ECMA's specifications.
- LiveScript and JScript were phased out as competing "languages".
- **JavaScript** adopted ECMA's rules and became the widely used implementation.
- From that point, **all new language features are defined by ECMAScript and implemented by JavaScript**.

CommonJS vs ECMAScript conventions

Two *conventions* for writing JavaScript still matter in practice (especially once Node.js enters the picture):

Aspect	CommonJS (older)	ECMAScript / ES6+ (modern)
Origin	Predates full modern ES standardization of modules/classes in the way we use	Modern standard

Aspect	CommonJS (older)	ECMAScript / ES6+ (modern)
	them today	
Preference for new apps	Legacy / still common in older Node code	**Always preferred** for new applications
Classes	No native `class` keyword; classes simulated with functions and prototypes	Native `class`, inheritance syntax, arrow functions, template literals, promises, and more (from ES6 onward)

The lecture's memorable jab at the pre-`class` world:

> "Think about what level of jugaad was happening: implementing a class through a function"

ES6 was the major leap; ES7, ES8, ES9 (and later yearly releases) added further enhancements. The evening session's demos still use **CommonJS** `require` / `exports` for modules; the next session is flagged to cover the ECMAScript `import` / `export` equivalent.

> **Exam:** ECMAScript = specification/blueprint; JavaScript = implementation. Do not call ECMAScript "a language" and stop.

> **Pitfall:** Confusing "JavaScript invented arrow functions" with "ECMAScript specified arrow functions, then engines implemented them."

Node.js: Runtime, Installation and First Run

The problem Node.js solves

Before Node.js, JavaScript lived **inside the browser**. You put a `<script>` tag in an HTML file; the browser's JS engine ran that script. There was **no independent way** to run a `.js` file on its own, and therefore no practical path to write **server-side** programs in JavaScript the way you write servers in Java, Python, or C++.

Mechanism: what Node.js is (and is not)

Node.js is a runtime environment and interpreter for JavaScript. Analogy from the lecture:

> "What JRE or JVM is for Java, Node.js is for JavaScript."

Claim	True or false?
Node.js is a language	**False**
Node.js is a library	**False**
Node.js is a framework	**False**
Node.js is software that runs your JavaScript	**True**

How it was built (conceptual model from the lecture):

1. Take the **JS engine** that already lived in the browser.
2. Extract / reuse that engine outside the browser.
3. Add more features so JS can do filesystem work, networking, etc. on a machine.
4. The engine Node uses is **V8** — the same engine family used by Google Chrome. Node.js itself is a **separate, independent project**; V8 is not "officially released by JavaScript" as a product you install alone for this purpose.

With Node.js installed, JavaScript can run **like C, C++, Java, or Python**:

- Create a `.js` file.
- Write code directly — **no** `<script>` tag, **no** HTML file, **no** browser required.
- Run: `node filename.js`
- That unlocks complete **servers and backend applications** in JavaScript.

```
Before Node.js                    After Node.js
──────────────                    ─────────────
HTML file                         plain .js file
   |                                   |
   v                                   v
<script> ... </script>            node demo.js
   |                                   |
   v                                   v
Browser JS engine                 Node.js (V8 + extras)
   |                                   |
   v                                   v
UI in the tab                     server / CLI / tools
```

Installing and verifying Node.js

- Official site: **nodejs.org** (tagline recalled in class: "run JavaScript everywhere").
- **Recommended versions in the lecture: v22 or v24**.
- **v18 is deprecated**; industry has largely moved to v22.
- Avoid the absolute latest (example given: v26.x) — may contain bugs or security issues.

- Stable-choice rule stated in class: prefer **one version before the absolute latest**.
- Analogy used: a recent PHP vulnerability as a reminder that "newest" is not always "safest"; Node has also seen library-level security issues.

Installation is a normal installer flow (Next → Accept Terms → Next → Install). Then verify in a terminal:

Command	What you should see
`node`	Welcome / REPL-style entry showing version context
`node --version` or `node -v`	Printed installed version

If Windows says **"node is not recognized as an internal or external command"**, the **PATH** was not set (or not refreshed). Fix the same way you fix a missing JDK path: Advanced System Settings → Environment Variables → Path → add the Node.js install directory.

Running a file:

1. Open a terminal in the folder that contains the `.js` file (Windows 10/11 shortcut: right-click the folder → **Open in Terminal**).
2. Run: `node demo.js`

> **Exam:** Node.js = runtime/interpreter for JS; not a language, library, or framework. It uses the V8 engine and lets you run `.js` files without a browser.

> **Pitfall:** Thinking "I wrote JavaScript in a `.js` file, so I can double-click it like HTML." Without Node (or another runtime), a bare `.js` file has no browser page to host it.

Modules: Concept, Types and require

The problem modules solve

A non-trivial program needs reusable bundles of behaviour — reading files, speaking HTTP, doing crypto — without pasting thousands of lines into every script. Languages already solved this with **header files** (C/C++) or **packages** (Java). Node.js's answer is the **module**.

Definition

A **module** is a collection of predefined functions, classes, properties, and constants — "simply a bundle or package containing certain functions, classes, properties."

Two types of modules in Node.js

Type	Meaning	Examples
Predefined (built-in)	Already available inside Node.js by default	`fs`, `http`, `path`, `os`, `crypto`, `events`, `stream`, `buffer`, `URL`, `child_process`, `util` (full list in Node docs)
User-defined (custom)	Created by you for the project	e.g. `mymodule.js` exporting `sum` / `product`

Loading a module with `require()`

Using any module starts with `require()` — same role as `#include` in C/C++ or `import` in Java:

```
const fs = require('fs');
```

What `require` does:

1. Loads the module.
2. Returns a **reference** to it.
3. You call functions **through that reference**: `fs.writeFile( ... )`, `http.createServer( ... )`, `myModule.sum( ... )`.

Naming convention from the lecture: name the variable the **same as the module** (`fs`, `http`) for readability. Any name works (`xyz`, `abc`), but matching names keep code clear. **Without the reference, you cannot call the module's functions.**

Built-in	Custom
`require('fs')` — name only	`require('./mymodule.js')` — **path** with `./` for current directory

Omitting `./` for a local file typically yields **Cannot find module**.

> **Exam:** Module = bundle of functions/classes/constants. Built-in vs user-defined. `require` returns a reference; calls go through that reference.

---

fs Module: Reading and Writing Files

The problem

Backend work constantly needs to persist and read data on disk. The `fs` **(file system) module** handles reading, writing, deleting, and renaming files.

Writing files: async vs sync

Asynchronous: `fs.writeFile`

```
const fs = require('fs');

console.log('before writing into file');
fs.writeFile('demo.txt', 'Hello from Node', function (err) {
  if (err) {
    console.log(err);
  } else {
    console.log('write operation completed');
  }
});
console.log('after writing file');
```

Rules emphasised in class:

- The **callback is mandatory**. Omitting it throws something in the spirit of: **cb argument must be of type function**.
- The callback fires **once the write finishes**.
- If the file does not exist, it is **created**; if it exists, content is **overwritten**.

Observed output order in the async demo:

1. `before writing into file`
2. `after writing file`
3. `write operation completed`

That order is the proof: the rest of the script continued **while** the write was still in progress.

Synchronous: `fs.writeFileSync`

```
const fs = require('fs');

fs.writeFileSync('demo.txt', 'Hello sync');
console.log('write finished, then this line runs');
```

- No callback.
- Execution **blocks** until the write completes.
- Output sequence is guaranteed in source order.

API	Blocking?	Callback?	Typical use
`fs.writeFile`	No (async)	Required	Servers, non-blocking I/O
`fs.writeFileSync`	Yes	No	Simple scripts, startup config

Reading files: async vs sync

Asynchronous: `fs.readFile`

```
const fs = require('fs');

console.log('before reading file');
fs.readFile('demo.txt', function (err, data) {
  if (err) {
    console.log(err);
  } else {
    console.log(data.toString());
  }
});
console.log('after reading file');
```

Callback shape: `(err, data)`.

- On success: `err` is `null`, `data` holds the contents.
- Recommended pattern: `if (err) { … } else { … }`.

Async demo order: `after reading file` **appeared first**; file content appeared **after** — again proving non-blocking behaviour.

Synchronous: `fs.readFileSync`

```
const fs = require('fs');
const data = fs.readFileSync('demo.txt');
console.log(data.toString());
```

Returns data directly; no callback.

Buffer vs string

File data from these APIs arrives as a **Buffer** (raw bytes), not a ready-made string — same idea as **byte streams** in Java file handling. Convert with:

```
data.toString()
```

> **Exam:** Async `writeFile` / `readFile` need callbacks; Sync variants block and return/complete in order. File bytes come as Buffer → use `toString()`.

> **Pitfall:** Expecting `console.log(data)` after `readFile` to print readable text without `toString()` — you will see Buffer notation instead of the plain string.

HTTP Module: Building a Local Server

The problem

Browsers speak **HTTP**. To serve responses from your machine without a browser-only page, Node's `http` **module** creates and runs an HTTP server process.

Step-by-step server creation

```
const http = require('http');

const server = http.createServer(function (request, response) {
  response.write('Hello World');
  response.end();
});

server.listen(6500);
console.log('Server started on port 6500');
```

Unpack each step:

1. `require('http')` — load the module.
2. `http.createServer(callback)` — creates a server object and returns a **reference**. The callback receives `(request, response)` : - **First argument** = incoming **request** object. - **Second argument** = outgoing **response** object. - Names can be anything (`req` / `res` , `x` / `y`); **position** decides meaning.
3. `server.listen(portNumber)` — starts listening for requests on that port.

createServer vs listen (critical distinction)

Call	What it does	Alone, what happens?
`createServer( ... )`	Builds the server object	Program can still **exit immediately** — nothing is listening
`listen(port)`	Starts the server and keeps the process alive	Server runs until you stop it (**Ctrl+C**)

Demo takeaway from class: without `listen()`, the program runs and exits. With `listen()`, it stays up indefinitely.

When does the createServer callback run?

Not when the `.js` file is first executed. It runs **only when a client sends a request** to the server.

> "This callback will only be executed when there is a request from a client onto this server."

Sending a response:

- `response.write(data)` — what to put in the body.
- `response.end()` — processing finished; send/finish the response.

Class demo: server on port **6500**; browser opened `127.0.0.1:6500` and received **Hello World**.

IP addresses, port numbers, and sockets

Term	Meaning
IP address	Uniquely identifies a **system** on a network. No network → no IP. IP can change across networks. Local machine loopback: `127.0.0.1` (also called **localhost**).
Port number	Uniquely identifies a **process/application** on that system. One machine can run e-commerce, social, DB, etc. at once — IP alone is not enough.
Socket	**IP + port**, e.g. `127.0.0.1:6500` — full address of one application on one host.

Lecture analogy:

> "IP address = railway station; port number = platform number."

Without the platform number, you cannot catch the right train.

Ports to avoid (often already occupied): `3306` (MySQL), `8080`, `8090`, `3000`, `4200` (React/Angular), `8000`, many **7000**-series. Occupied port → **address already in use**. The demo chose **6500** to dodge conflicts.

```
  Client browser
       |
       |  HTTP request to 127.0.0.1:6500
       v
+--------------------+
| Your PC            |
|  IP: 127.0.0.1     |
|  +-------------+   |
|  | Port 6500   |<------ Node http server (this process)
|  +-------------+   |
|  +-------------+   |
|  | Port 3306   |<------ (example) MySQL, if running
|  +-------------+   |
+--------------------+
```

Exam: `createServer` creates; `listen` starts and holds the process. Callback fires on **request**, not on startup. Socket = IP + port. `127.0.0.1` = local machine.

Pitfall: Forgetting `response.end()` — client may hang waiting for the response to finish. Forgetting `listen()` — "server" object exists but nothing accepts connections.

Custom Modules: Creating and Exporting

The problem

You can put helpers in a second `.js` file, but **functions defined in one file are not visible in another by default**. Even `require`-ing the file is not enough unless those functions are **exported**.

> "A function defined in one file can only be called in that same file."

Creating and exporting (CommonJS style used in class)

mymodule.js — define and export:

```
exports.sum = function (a, b) {
  return a + b;
};

exports.product = function (a, b) {
```

```
  return a * b;
};
```

Each export is attached as a named property: `exports.functionName = function ( ... ) { ... }`.

app.js — require by **path**, then call through the reference:

```
const myModule = require('./mymodule.js');

console.log(myModule.sum(2, 5));
console.log(myModule.product(3, 8));
```

Kind	`require` argument
Built-in	`'fs'`, `'http'` — bare name
Custom file in same folder	`'./mymodule.js'` — **must** include `./` (or a proper relative/absolute path)

Omit `./` → typically **Cannot find module**.

> **Exam:** Custom modules need `exports.…` (CommonJS as taught today) plus `require('./file.js')`. Built-ins use name-only require.

> **Pitfall:** Writing functions in `mymodule.js` and expecting `require('./mymodule.js')` to magically expose them without `exports`. Also: mixing this up with upcoming ES `import` / `export` syntax — today's demos are CommonJS.

Combined Worked Example: Math Module + File Log + HTTP Server

This single mini-project ties together **custom modules**, `fs`, and `http`: a tiny server that uses exported math helpers and appends each hit to a log file.

File: `calc.js` (user-defined module)

```
exports.sum = function (a, b) {
  return a + b;
};
```

```
exports.product = function (a, b) {
  return a * b;
};
```

File: `server.js` (entry point)

```
const http = require('http');
const fs = require('fs');
const calc = require('./calc.js');

const PORT = 6500;

const server = http.createServer(function (request, response) {
  const a = 2;
  const b = 5;
  const sum = calc.sum(a, b);
  const product = calc.product(a, b);

  const line =
    'Request received. sum(' + a + ',' + b + ')=' + sum +
    ' product(' + a + ',' + b + ')=' + product + '\n';

  fs.writeFile('hits.log', line, function (err) {
    if (err) {
      response.write('Error writing log');
      response.end();
      return;
    }

    response.write('Hello from Node server\n');
    response.write(line);
    response.end();
  });
});

server.listen(PORT);
console.log('Listening on http://127.0.0.1:' + PORT);
```

How to run

1. Put both files in the same folder.
2. Open a terminal in that folder.
3. Run: `node server.js`
4. Browse to `http://127.0.0.1:6500`
5. Stop the server with Ctrl+C when done.

End-to-end trace

1. Node loads `server.js`. `require('http')` and `require('fs')` resolve **built-in** modules by name. `require('./calc.js')` loads your file; only `exports.sum` and `exports.product` are visible on `calc`.
2. `createServer` builds the server; `listen(6500)` binds the **socket** `127.0.0.1:6500` (loopback IP + chosen port) and keeps the process alive.
3. The createServer callback does **not** run yet.
4. Browser requests that socket. Node invokes the callback with `(request, response)`.
5. `calc.sum(2, 5)` → `7`; `calc.product(2, 5)` → `10` (here product uses 2 and 5 for a single consistent demo; the lecture's separate product example was `3, 8` → `24`).
6. `fs.writeFile` starts an **async** write of `hits.log`. Other work could continue; here we wait in the callback before finishing the HTTP response so the client sees success only after the log attempt finishes.
7. `response.write` + `response.end` send the body back to the browser.
8. Without `listen`, step 2 never happens and the process exits. Without `exports` on `calc.js`, step 5 fails. Without `./` in `require('./calc.js')`, module load fails.

Quick Revision Checklist

Topic	One-line takeaway
ECMAScript	Blueprint/specification; JavaScript implements it; yearly feature releases.
Browser wars	LiveScript vs JScript → ECMA unified the rules (~1996–97).
CommonJS vs ES	Older vs modern conventions; new apps prefer ES6+; today's module demos still use CommonJS.
Node.js	Runtime/interpreter for JS (like JVM/JRE for Java); not a language/library/framework; uses V8.
Install / PATH	Prefer stable versions (lecture: v22/v24); fix PATH if `node` is unrecognized; run with `node file.js`.
Module	Bundle of functions/classes/constants; built-in vs user-defined.
require	Loads module, returns reference; call APIs through that reference.
fs write/read	Async needs callback (non-blocking order); Sync blocks; data is Buffer → `toString()`.

Topic	One-line takeaway
http server	`createServer` builds; `listen` starts; callback runs on **client request**.
Socket	IP + port; `127.0.0.1` is localhost; avoid busy ports (3306, 3000, 8080, …).
Custom module	`exports.name = function…` and `require('./file.js')`.

Practice Exercises

Concept

1. In one sentence each, distinguish **ECMAScript** from **JavaScript**.
2. List three things Node.js is **not**, according to the lecture's framing.
3. What is a **socket** in the sense used in this lecture?

Application

4. Write a complete `greet.js` that exports a function `hello(name)` returning `"Hello, " + name`. Write `main.js` that requires it correctly and prints `hello("WPT")`.
5. Write a minimal HTTP server on port `6500` that responds with `OK` and ends the response. State what happens if you omit `listen`.

Predict the output

6. Predict the **order** of console lines:

```
const fs = require('fs');
console.log('A');
fs.writeFile('t.txt', 'x', function () {
  console.log('B');
});
console.log('C');
```

7. Predict whether the createServer callback prints `hit` when you only run `node server.js` with no browser request:

```
const http = require('http');
http.createServer(function () {
  console.log('hit');
}).listen(6500);
console.log('started');
```

Spot the bug

8. Find and fix the bugs:

```
const fs = require('fs');
fs.writeFile('a.txt', 'hi');
const data = fs.readFile('a.txt');
console.log(data.toString());
```

9. Find and fix the bugs:

```
// utils.js
function add(a, b) {
  return a + b;
}

// app.js
const utils = require('utils.js');
console.log(utils.add(1, 2));
```

Practice Answers

1. ECMAScript is the **specification/blueprint** (rules and features). JavaScript is the **implementation** that follows that blueprint.

2. Node.js is **not** a language, **not** a library, and **not** a framework — it is a **runtime/interpreter** that runs JavaScript (analogous to JVM/JRE for Java).

3. A **socket** is the combination of an **IP address + port number** (e.g. `127.0.0.1:6500`) that identifies a specific application on a specific machine.

4.

```
// greet.js
exports.hello = function (name) {
  return 'Hello, ' + name;
};
```

```
// main.js
const greet = require('./greet.js');
console.log(greet.hello('WPT'));
```

5.

```
const http = require('http');
const server = http.createServer(function (request, response) {
  response.write('OK');
  response.end();
});
server.listen(6500);
```

If you omit `listen`, the server object may be created but the process typically **exits immediately** and nothing accepts connections.

6. Console order: `A`, then `C`, then `B`. `writeFile` is asynchronous, so `C` runs before the write callback prints `B`.

7. On startup you see `started`. You do **not** see `hit` until a client actually requests the server. The createServer callback runs on request, not on file load.

8. Two issues: (1) async `writeFile` **requires a callback**; (2) async `readFile` does **not** return data directly — it takes a callback `(err, data)`. A corrected async version:

```
const fs = require('fs');
fs.writeFile('a.txt', 'hi', function (err) {
  if (err) {
    console.log(err);
    return;
  }
  fs.readFile('a.txt', function (err2, data) {
    if (err2) {
      console.log(err2);
      return;
    }
    console.log(data.toString());
  });
});
```

(Alternatively use `writeFileSync` / `readFileSync` if blocking is acceptable.)

9. Two issues: (1) `add` was never **exported**; (2) local file require needs `./utils.js`, not `'utils.js'`. Fix:

```
// utils.js
exports.add = function (a, b) {
  return a + b;
};
```

```
// app.js
const utils = require('./utils.js');
```

```
console.log(utils.add(1, 2));
```